

The Vulnerability Scanner (Static Analyzer)

Team Members:

1)Menna allah Mostafa	192300166
2) Basma yasser	192300538
3)Mai Ashraf	192300095
4)Malak Mohamed	192300094
5)Gana Ahmed	192300067

Instructor name:Dr. Hossam reda

Teaching Assistant :Eng. Mohamed khatab, Eng. Asmaa Elboghaday

Introduction

In the realm of cybersecurity, **Static Application Security Testing (SAST)** is a critical first line of defense. This tool, titled **The Vulnerability Scanner**, is a lightweight yet powerful static analyzer designed to scan Python source code for security vulnerabilities and logical inconsistencies without ever executing the script. By analyzing the "skeleton" of the code, it identifies risks before they can be exploited in a live environment.

Project Overview

The primary goal of this project is to automate the detection of common security pitfalls. It focuses on three main areas:

- **Arbitrary Code Execution:** Identifying dangerous functions that could allow attackers to run malicious commands.
- **Credential Security:** Detecting hardcoded passwords within the source code.
- **Code Hygiene:** Finding unused variables to reduce technical debt and potential logic errors.

The scanner provides a dual-reporting system: a **color-coded terminal output** for immediate feedback and a **persistent text file report** for archival and auditing purposes.

How It Works: The "Compiler" Approach

This scanner replicates the early stages of a compiler to understand the code's intent rather than just searching for keywords.

The Abstract Syntax Tree (AST)

Instead of using simple "find and replace" (which can be easily fooled), this tool uses the `ast` module to convert Python code into an **Abstract Syntax Tree**.

- **`ast.parse(code)`:** This function acts as the **Lexer/Parser**. It breaks down raw text into a hierarchical tree of "Nodes" (e.g., Function Calls, Assignments, Attributes).

- **ast.walk(tree):** This acts as the **Tree Traverser**. It visits every "leaf" of the tree to inspect the logic.

Logic Implementation

The script replicates compiler-like tracking through specific node inspections:

Variable Tracking (ast.Assign & ast.Name):

- It creates a "Symbol Table" by storing variable names in the assigned dictionary.
- It monitors ast.Load contexts to see if a variable is ever actually used.

Function Call Inspection (ast.Call):

- It identifies Simple Calls (e.g., eval()) by checking node.func as a Name.
- It identifies Qualified Calls (e.g., os.system()) by checking node.func as an Attribute of a specific module.

Severity Mapping:

- **HIGH:** Direct threats to system integrity (eval, exec, os.system).
 - **MEDIUM:** Data privacy risks (Hardcoded variable names like password).
 - **LOW:** Optimization issues (Unused variables).
-

Code Input and Output Examples

```
import os
```

```
password = "admin_secret_123"
user_input = "ls -la"

def execute_task():
    data_chunk = 1024 # This variable is never used
    eval("print('Hello')")
    os.system(user_input)

execute_task()
```

Scanner Output

Upon running `python scanner.py vulnerable_file.py`, the tool generates the following:

Terminal View

Scanning: vulnerable_file.py

```
[MEDIUM] Hardcoded Password at line 3
[HIGH] Dangerous function eval() at line 8
[HIGH] Dangerous function os.system() at line 9
[LOW] Unused variable: data_chunk (defined at line 7)
File Report (report.txt)
===== SECURITY REPORT =====
Generated at: Tue May 5 20:11:10 2026
```

```
[MEDIUM] Hardcoded Password at line 3
[LOW] Unused variable: data_chunk (defined at line 7)
[HIGH] Dangerous function eval() at line 8
[HIGH] Dangerous function os.system() at line 9

===== END REPORT =====
```

Conclusion

The Vulnerability Scanner demonstrates the power of programmatic code analysis. By leveraging Python's AST capabilities, it moves beyond basic

text searching to understand the structural logic of a program. While this is a "static" tool and cannot catch every runtime edge case, it serves as an essential utility for developers to ensure their code follows basic security practices and maintains a clean, efficient structure.

Future Enhancements:

- Adding support for SQL injection detection in string formatting.
- Integration with CI/CD pipelines to block insecure commits.
- Support for custom user-defined "dangerous" function lists.